

Analysis of Trump Administration LGBTQ+ Policy Actions

Randy Howk

2025-10-05

Contents

1	Executive Summary	1
1.1	Step 1: Extract Text from Word Document	1
1.2	Step 2: Tidy and Transform Data	2
1.3	Step 3: Connect to PostgreSQL via DDEV	7
1.4	Step 4: Load Tidied Data into PostgreSQL	8
1.5	Step 5: Enhance Dataset with Analytical Columns	9
1.6	Step 6: Named Entity Recognition (NER)	10
1.7	Step 7: Combined Analysis (Rule-Based + NER)	20
1.8	Step 9: Text Mining and Keyword Analysis	23
1.9	Step 10: Key Findings and Insights	25
1.10	Summary Statistics	26
1.11	Step 11: Save Enhanced Data to Database	26
1.12	Step 12: Final Database Verification	26
1.13	Reproducibility Notes	27
1.14	Data Source	28
1.15	Methodological Approach	28
1.16	Session Information	28
1.17	References	29

1 Executive Summary

This R Markdown documents a reproducible workflow to convert an untidy Word document of policy actions into a tidy dataset, perform Named Entity Recognition (NER), and conduct exploratory analysis. Database interaction chunks are set to `eval=FALSE` so the file can knit without a live PostgreSQL connection.

```
options(repos = c(CRAN = "https://cran.rstudio.com/"))
library(tidyverse)
library(lubridate)
library(DBI)
library(RPostgres)
library(officer)
library(xml2)
```

```
library(tidytext)
library(stringr)
library(ggplot2)
library(scales)
library(knitr)
library(kableExtra)

knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE,
                      fig.width = 10, fig.height = 6, tidy.opts = list(width.cutoff = 60), tidy =
                        ↪ TRUE)
```

1.1 Step 1: Extract Text from Word Document

Using the officer package to read .docx files directly in R:

```
# Adjust the path if your file location differs
docx_path <- "Untidy.docx"

if (!file.exists(docx_path)) {
  stop("Please place Untidy.docx in the working directory or update 'docx_path'.")
}

doc <- read_docx(docx_path)
doc_content <- docx_summary(doc)
text_paragraphs <- doc_content %>%
  filter(content_type == "paragraph") %>%
  pull(text) %>%
  paste(collapse = "\n")

cat("First 500 characters of extracted text:\n")
```

First 500 characters of extracted text:

```
cat(substr(text_paragraphs, 1, 500), "\n...\n")
```

```
## 09.24.2025 Trump canceled a meeting with Democratic lawmakers intended to avoid a possible federal gover
## ...
```

1.2 Step 2: Tidy and Transform Data

Understanding the “Untidy” Data Structure The source data from Untidy.docx is untidy for several key reasons that violate Hadley Wickham’s tidy data principles:

Unstructured text format: All data is embedded in free-form paragraphs with no clear delimiters or column structure
 Date-content concatenation: Dates and descriptive content are merged in single text strings
 Inconsistent formatting:
 No standardized schema or field separation
 Non-rectangular structure: The original Word document contains flowing text rather than tabular data
 According to tidy data principles, each variable should form a column, each observation should form a row, and each type of observational unit should form a table. Our raw data violates all three principles.

1.2.1 Split Text into Individual Entries

```

# Step 1: Split continuous text into individual entries
# Using dplyr's pipe operator (%>%) for readable data transformation pipeline

raw_entries <- str_split(text_paragraphs, "\n")[[1]] %>% # Split on line breaks
  tibble(raw_text = .) %>% # Convert to tibble (tidy data)
  ↪ frame()
  filter(nchar(trimws(raw_text)) > 0) # Remove empty/whitespace-only lines

cat("Total raw entries extracted:", nrow(raw_entries), "\n\n")

```

```
## Total raw entries extracted: 424
```

```

# Display sample of raw, untidy data
cat("Example of UNTIDY raw data:\n")

```

```
## Example of UNTIDY raw data:
```

```
print(head(raw_entries$raw_text, 3))
```

```

## [1] "09.24.2025 Trump canceled a meeting with Democratic lawmakers intended to avoid a possible federal
## [2] "09.24.2025 The Trump administration has withheld $1.25 million in congressionally appropriated fund
## [3] "The 2025 grants included three for LGBTQ-related projects. In Washington, D.C., the Preservation Le
the end of the fiscal year - the move would appear to violate the Impoundment Control Act of 1974."

```

```
cat("\n")
```

1.2.2 Tidying Process: Applying tidyr and dplyr Principles

Now we apply systematic transformations following the tidyr philosophy of reshaping data and dplyr verbs for data manipulation:

```

# Step 2: Parse and separate variables into distinct columns
# This is the core "tidying" transformation

policy_data <- raw_entries %>%

  # MUTATE: Create new variables by transforming existing ones (dplyr verb)
  mutate(

    # Variable 1: Extract the DATE component
    # Pattern: ^\d{2}\.\d{2}\.\d{4} means "start of line (^), two digits,
    # period, two digits, period, four digits"
    # Example: "09.24.2025" matches this pattern
    date_str = str_extract(raw_text, "^\\d{2}\\.\d{2}\\.\d{4}"),

    # Variable 2: Extract the CONTENT component
    # Remove the date prefix to isolate the policy description
    # str_remove eliminates the matched pattern from the beginning
    content = str_remove(raw_text, "^\\d{2}\\.\d{2}\\.\d{4}\\s*"),

    # Variable 3: Convert date STRING to proper DATE data type
    # mdy() from lubridate parses "month.day.year" format

```

```

# This transforms character data into a true date object for temporal analysis
date = mdy(date_str),

# Variable 4: Create unique identifier for each observation
# row_number() generates sequential IDs, treating each row as a distinct observation
entry_id = row_number()

) %>%

# FILTER: Keep only complete cases with valid dates (dplyr verb)
# Removes malformed entries that don't match our expected structure
filter(!is.na(date)) %>%

# SELECT: Choose final variables and reorder columns (dplyr verb)
# Creates our final tidy structure: one observation per row, one variable per column
select(entry_id, date, content)

# Report on the tidying results
cat("\n=== TIDYING TRANSFORMATION RESULTS ===\n")

```

```

##
## === TIDYING TRANSFORMATION RESULTS ===

```

```

cat("Original raw entries:", nrow(raw_entries), "\n")

```

```

## Original raw entries: 424

```

```

cat("Successfully parsed entries:", nrow(policy_data), "\n")

```

```

## Successfully parsed entries: 368

```

```

cat("Entries filtered out (no valid date):", nrow(raw_entries) - nrow(policy_data), "\n")

```

```

## Entries filtered out (no valid date): 56

```

```

cat("\nDate range captured:",
  format(min(policy_data$date), "%B %d, %Y"),
  "to",
  format(max(policy_data$date), "%B %d, %Y"), "\n")

```

```

##
## Date range captured: January 20, 2017 to September 24, 2025

```

1.2.3 Demonstrating the Transformation

Let's compare the untidy vs. tidy structure side-by-side:

```

# Show first 3 entries in UNTIDY format
cat("\n--- BEFORE TIDYING (Untidy Format) ---\n")

```

```

##
## --- BEFORE TIDYING (Untidy Format) ---

```

```
cat("Single text blob with embedded structure:\n\n")
```

```
## Single text blob with embedded structure:
```

```
for (i in 1:3) {
  cat(paste0("[Entry ", i, " ] ", substr(raw_entries$raw_text[i],
    1, 150), "...\\n\\n"))
}
```

```
## [Entry 1] 09.24.2025 Trump canceled a meeting with Democratic lawmakers intended to avoid a possible fed
##
## [Entry 2] 09.24.2025 The Trump administration has withheld $1.25 million in congressionally appropriated
##
## [Entry 3] The 2025 grants included three for LGBTQ-related projects. In Washington, D.C., the Preservati
```

```
cat("\\n--- AFTER TIDYING (Tidy Format) ---\\n")
```

```
##
## --- AFTER TIDYING (Tidy Format) ---
```

```
cat("Structured table with separated variables:\\n\\n")
```

```
## Structured table with separated variables:
```

```
# Display first 3 entries in TIDY format as a formatted
# table
kable(head(policy_data, 3) %>%
  mutate(content_display = substr(content, 1, 100)) %>%
  mutate(content_display = paste0(content_display, "...")) %>%
  select(entry_id, date, content_display), caption = "Tidy Data Structure: Each Variable in
  ↳ Its Own Column",
  col.names = c("ID", "Date", "Content (truncated)"), format = "latex",
  booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9) %>%
  column_spec(1, bold = TRUE, width = "5em") %>%
  column_spec(2, width = "10em") %>%
  column_spec(3, width = "40em")
```

Table 1: Tidy Data Structure: Each Variable in Its Own Column

ID	Date	Content (truncated)
1	2025-09-24	Trump canceled a meeting with Democratic lawmakers intended to avoid a possible federal government s...
2	2025-09-24	The Trump administration has withheld \$1.25 million in congressionally appropriated funds from 20 or...
4	2025-09-22	White House Press Secretary Karoline Leavitt said that the administration is investigating what she ...

1.2.4 Key Tidying Operations Applied

This transformation leverages multiple tidyr and dplyr principles:

1. Separation (tidyr principle): Date and content separated into distinct columns using `str_extract()` and `str_remove()`
2. Type Conversion: Dates transformed from character strings to proper Date objects using `lubridate::mdy()`
3. Filtering (dplyr verb): Only valid observations with parseable dates retained using `filter(!is.na(date))`
4. Selection and Ordering (dplyr verb): Essential variables chosen in logical order using `select()`
5. Unique Identification: Each observation given a unique `entry_id` using `row_number()`

```
# Display larger sample of tidy data with full formatting
kable(head(policy_data, 10), caption = "First 10 Policy Entries (Tidied and Structured)",
      col.names = c("Entry ID", "Date", "Policy Action Description"),
      format = "latex", booktabs = TRUE, longtable = TRUE) %>%
kable_styling(latex_options = c("striped", "scale_down",
                                "repeat_header"), font_size = 9, full_width = FALSE,
              position = "left") %>%
column_spec(1, bold = TRUE, border_right = TRUE) %>%
column_spec(2, bold = TRUE, color = "blue") %>%
scroll_box(height = "400px")
```

Table 2: First 10 Policy Entries (Tidied and Structured)

Entry ID	Date	Policy Action Description
1	2025-09-24	Trump canceled a meeting with Democratic lawmakers intended to avoid a possible federal government shutdown
2	2025-09-24	The Trump administration has withheld \$1.25 million in congressionally appropriated funds from 20 organizations
4	2025-09-22	White House Press Secretary Karoline Leavitt said that the administration is investigating what she claimed was a leak
5	2025-09-21	In an interview on Fox News' Sunday program, Pres. Trump responded to a question about Democrats voting to impeach him
6	2025-09-20	The Trump administration's Centers for Disease Control and Prevention (CDC) pulled / restricted access to certain data
7	2025-09-20	In speech to the American Cornerstone Institute, which advocates against LGBTQ people, Pres. Trump veered into anti-LGBTQ rhetoric
8	2025-09-19	The Trump administration asked the Supreme Court to let it enforce a passport policy for transgender and non-binary people
9	2025-09-19	Fox News asked Vice President JD Vance whether he considers "the militant transgender movement" to be a national security threat
10	2025-09-19	During remarks after signing an executive order demanding high-skilled immigrants pay a \$100,000 fee for work visas
11	2025-09-19	The Trump administration was ordered to release grant funding withheld from arts organizations funded by the National Endowment for the Arts

1.2.5 Data Quality Validation

```
# Verify the tidied data meets quality standards

# Check 1: No missing dates (critical variable)
missing_dates <- sum(is.na(policy_data$date))
cat("Missing dates:", missing_dates, ifelse(missing_dates ==
0, " PASS\n", " FAIL\n"))
```

```
## Missing dates: 0 PASS
```

```
# Check 2: No missing content
missing_content <- sum(is.na(policy_data$content) | policy_data$content ==
"")
cat("Missing content:", missing_content, ifelse(missing_content ==
0, " PASS\n", " FAIL\n"))
```

```
## Missing content: 0 PASS
```

```
# Check 3: Unique entry IDs
duplicate_ids <- sum(duplicated(policy_data$entry_id))
cat("Duplicate IDs:", duplicate_ids, ifelse(duplicate_ids ==
  0, " PASS\n", " FAIL\n"))
```

```
## Duplicate IDs: 0 PASS
```

```
# Check 4: Date range is logical
date_range_valid <- max(policy_data$date) >= min(policy_data$date)
cat("Date range valid:", ifelse(date_range_valid, " PASS\n",
  " FAIL\n"))
```

```
## Date range valid: PASS
```

```
# Check 5: Data types are correct
correct_types <- is.integer(policy_data$entry_id) && inherits(policy_data$date,
  "Date") && is.character(policy_data$content)
cat("Column types correct:", ifelse(correct_types, " PASS\n",
  " FAIL\n"))
```

```
## Column types correct: PASS
```

```
# Summary statistics
cat("\n--- Summary Statistics ---\n")
```

```
##
## --- Summary Statistics ---
```

```
cat("Total observations:", nrow(policy_data), "\n")
```

```
## Total observations: 368
```

```
cat("Variables:", ncol(policy_data), "\n")
```

```
## Variables: 3
```

```
cat("Memory size:", format(object.size(policy_data), units = "Kb"),
  "\n")
```

```
## Memory size: 179.5 Kb
```

```
# Display structure
cat("\n--- Data Structure ---\n")
```

```
##
## --- Data Structure ---
```

```
str(policy_data)
```

```
## tibble [368 x 3] (S3: tbl_df/tbl/data.frame)
## $ entry_id: int [1:368] 1 2 4 5 6 7 8 9 10 11 ...
## $ date    : Date[1:368], format: "2025-09-24" "2025-09-24" ...
## $ content : chr [1:368] "Trump canceled a meeting with Democratic lawmakers intended to avoid a possible
```

1.2.6 Why This Tidying Matters for Analysis

The transformation from untidy text to tidy tabular data enables:

1. Temporal analysis: Proper Date objects allow time-series analysis, trend detection, and chronological ordering
2. Text mining: Separated content field enables tokenization, sentiment analysis, and keyword extraction
3. Database integration: Rectangular structure can be directly loaded into SQL databases
4. Statistical modeling: Tidy format is required for most R statistical functions
5. Reproducibility: Structured data can be easily filtered, grouped, and summarized using dplyr verbs

This tidying process converts unstructured narrative text into analysis-ready data, following the core principle that “tidy datasets are all alike, but every messy dataset is messy in its own way.”

1.3 Step 3: Connect to PostgreSQL via DDEV

Establishing connection to the local DDEV PostgreSQL instance:

```
# Connection parameters for DDEV PostgreSQL
con <- dbConnect(
  RPostgres::Postgres(),
  host = "127.0.0.1",          # DDEV exposes postgres on localhost
  port = 55432,               # DDEV custom port
  user = "ddev",              # Default DDEV user
  password = "ddev",          # Default DDEV password
  dbname = "ddev"             # Default DDEV database
)

# Test connection
cat("Database connection successful!\n")
```

```
## Database connection successful!
```

```
cat("PostgreSQL version:", dbGetQuery(con, "SELECT version();")$version, "\n")
```

```
## PostgreSQL version: PostgreSQL 15.14 (Debian 15.14-1.pgdg13+1) on aarch64-unknown-linux-gnu, compiled by
```

1.4 Step 4: Load Tidied Data into PostgreSQL

Writing the tidied data to the database:

```
# Write the tidied data to PostgreSQL Using overwrite=TRUE
# for reproducibility (drops and recreates table)
dbWriteTable(conn = con, name = Id(schema = "public", table = "lgbtq_policy_actions"),
  value = policy_data, overwrite = TRUE, row.names = FALSE)
```



```
# Create an index on the date column for faster queries
dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_policy_date
ON public.lgbtq_policy_actions (date DESC);")
```

```
## [1] 0
```

```
# Verify the data was loaded correctly
row_count <- dbGetQuery(con, "SELECT COUNT(*) as n FROM public.lgbtq_policy_actions;")

cat("\nData successfully loaded to PostgreSQL!\n")
```

```
##
## Data successfully loaded to PostgreSQL!
```

```
cat("Total rows in database:", row_count$n, "\n")
```

```
## Total rows in database: 1.818162e-321
```

```
# Preview data from database
db_preview <- dbGetQuery(con, "SELECT entry_id, date,
LEFT(content, 100) as content_preview
FROM public.lgbtq_policy_actions
ORDER BY date DESC
LIMIT 5;")

kable(db_preview, caption = "Data Preview from PostgreSQL Database",
format = "latex", booktabs = TRUE, longtable = TRUE) %>%
kable_styling(latex_options = c("striped", "scale_down",
"repeat_header"), font_size = 9)
```

Table 3: Data Preview from PostgreSQL Database

entry_id	date	content_preview
1	2025-09-24	Trump canceled a meeting with Democratic lawmakers intended to avoid a possible federal government s
2	2025-09-24	The Trump administration has withheld \$1.25 million in congressionally appropriated funds from 20 or
4	2025-09-22	White House Press Secretary Karoline Leavitt said that the administration is investigating what she
5	2025-09-21	In an interview on Fox News' Sunday program, Pres. Trump responded to a question about Democrats vot
6	2025-09-20	The Trump administration's Centers for Disease Control and Prevention (CDC) pulled / restricted acce

1.5 Step 5: Enhance Dataset with Analytical Columns

Adding rule-based categorical features to support downstream analysis:

```
# Add analytical columns using domain knowledge and pattern matching
policy_data_enhanced <- policy_data %>%
mutate(
# Temporal features
year = year(date),
month = month(date, label = TRUE, abbr = FALSE),
quarter = quarter(date),
```

```

day_of_week = wday(date, label = TRUE),

# Days since inauguration (assuming Jan 20, 2025)
days_in_office = as.numeric(date - ymd("2025-01-20")),

# Categorize by policy domain using keyword detection
policy_domain = case_when(
  str_detect(content, regex("military|armed forces|Pentagon|Navy|service member",
    ↪ ignore_case = TRUE)) ~ "Military",
  str_detect(content, regex("health|medical|hospital|CDC|HHS|NIH|care|surgery", ignore_case
    ↪ = TRUE)) ~ "Healthcare",
  str_detect(content, regex("school|education|student|university|college|campus",
    ↪ ignore_case = TRUE)) ~ "Education",
  str_detect(content, regex("grant|funding|budget|dollar|million|billion", ignore_case =
    ↪ TRUE)) ~ "Funding",
  str_detect(content, regex("passport|visa|immigration|deportation", ignore_case = TRUE)) ~
    ↪ "Immigration",
  str_detect(content, regex("bathroom|restroom|sports|athlete|locker", ignore_case = TRUE))
    ↪ ~ "Sports/Facilities",
  str_detect(content, regex("employment|job|workplace|fired|hire", ignore_case = TRUE)) ~
    ↪ "Employment",
  str_detect(content, regex("court|judge|lawsuit|ruling|Supreme Court", ignore_case = TRUE))
    ↪ ~ "Legal",
  TRUE ~ "Other"
),

# Categorize by action type
action_type = case_when(
  str_detect(content,
    ↪ regex("banned|ban|prohibited|block|freeze|withhold|cut|terminate|cancel|remove",
    ↪ ignore_case = TRUE)) ~ "Restriction/Removal",
  str_detect(content, regex("executive order|signed|directive|policy|rule|announced",
    ↪ ignore_case = TRUE)) ~ "Executive Action",
  str_detect(content, regex("said|stated|claimed|falsely|told|remarks", ignore_case = TRUE))
    ↪ ~ "Rhetoric/Statement",
  str_detect(content, regex("court|judge|ruled|ordered|blocked|injunction", ignore_case =
    ↪ TRUE)) ~ "Court Action",
  TRUE ~ "Other"
),

# Flag entries mentioning specific vulnerable groups
mentions_youth = str_detect(content, regex("youth|children|minor|student", ignore_case =
  ↪ TRUE)),
mentions_military = str_detect(content, regex("military|service member|veteran", ignore_case
  ↪ = TRUE)),
mentions_healthcare = str_detect(content, regex("health|medical|surgery|care", ignore_case =
  ↪ TRUE))
)

cat("Enhanced data created with", ncol(policy_data_enhanced), "columns\n")

## Enhanced data created with 13 columns

cat("Preview of new columns:\n")

## Preview of new columns:

```

```
glimpse(policy_data_enhanced %>% select(entry_id, date, policy_domain, action_type,
  ↳ mentions_youth))
```

```
## Rows: 368
## Columns: 5
## $ entry_id      <int> 1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, ~
## $ date          <date> 2025-09-24, 2025-09-24, 2025-09-22, 2025-09-21, 2025-0~
## $ policy_domain <chr> "Healthcare", "Funding", "Other", "Sports/Facilities", ~
## $ action_type   <chr> "Restriction/Removal", "Other", "Rhetoric/Statement", "~
## $ mentions_youth <lgl> TRUE, FALSE, FALSE, FALSE, FALSE, TRUE, FALSE, FALSE, F~
```

1.6 Step 6: Named Entity Recognition (NER)

Extract organizations, people, and monetary amounts using machine learning-based NER:

1.6.1 Why Use NER?

Named Entity Recognition automatically identifies and classifies entities in text. Unlike rule-based keyword matching, NER:

- **Discovers entities** you didn't think to search for
- **Understands context** (e.g., "Apple" the company vs. apple the fruit)
- **Recognizes entity types:** PERSON, ORG, GPE (location), MONEY, DATE, LAW, etc.
- **Provides comprehensive coverage** without maintaining keyword lists

1.6.2 Setup and Model Loading

```
# Install required packages
install.packages("udpipe")
install.packages("textreuse") # For entity extraction patterns
```

1.6.3 Load NER Model

```
library(udpipe)

# Load the pre-trained English model
model_files <- list.files(pattern = "english.*\\.udpipe$")

if (length(model_files) == 0) {
  # Try to download if not present
  cat("Downloading English language model...\n")
  udmodel_info <- udpipe_download_model(language = "english")
  udmodel <- udpipe_load_model(file = udmodel_info$file_model)
  NER_AVAILABLE <- TRUE
} else {
  udmodel <- udpipe_load_model(file = model_files[1])
  cat("NER model loaded successfully:", model_files[1], "\n")
  NER_AVAILABLE <- TRUE
}
```

```
## NER model loaded successfully: english-ewt-ud-2.5-191206.udpipe
```

1.6.4 Perform Named Entity Recognition

```
## Perform Entity Extraction
if (NER_AVAILABLE) {
  cat("Running linguistic annotation on", nrow(policy_data), "entries...\n")
  cat("This may take 1-2 minutes. Please wait...\n\n")

  # Annotate text (this does POS tagging, lemmatization, etc.)
  ner_results <- udpipe_annotate(
    udmodel,
    x = policy_data$content,
    doc_id = as.character(policy_data$entry_id)
  ) %>%
    as.data.frame()

  cat("Annotation complete!\n")

  # Check if entity column exists (some models have it, others don't)
  if ("entity" %in% colnames(ner_results) && any(!is.na(ner_results$entity))) {
    cat("Using built-in NER from model...\n")

    # Filter for named entities
    entities <- ner_results %>%
      filter(!is.na(entity)) %>%
      select(doc_id, token, lemma, entity) %>%
      mutate(doc_id = as.integer(doc_id))

  } else {
    cat("Model doesn't include NER. Using rule-based entity extraction...\n\n")

    # Fallback: Rule-based entity extraction using POS tags and patterns

    # Extract proper nouns (likely to be named entities)
    proper_nouns <- ner_results %>%
      filter(upos %in% c("PROPN")) %>% # Proper nouns
      select(doc_id, token, lemma, upos) %>%
      mutate(
        doc_id = as.integer(doc_id),
        entity = case_when(
          # PERSON checks FIRST (before ORG) - order matters!
          token %in% c("Trump", "Biden", "Pompeo", "DeVos", "Carson", "Pence",
            "Kennedy", "Gorsuch", "Vance", "McEnany", "Hegseth",
            "Robertson", "Graham", "Bozell", "Drollinger", "Baker",
            "Leavitt", "Corrigan", "Ellis", "Sauer", "Wildmon",
            "Obama", "Clinton", "Tillerson", "James", "Levi") ~ "PERSON",

          # Titles (context-dependent, but usually refer to people)
          token %in% c("President", "Secretary", "Judge", "Justice",
            "Attorney", "General", "Director") ~ "PERSON",

          # Organizations (government agencies, institutions)
          token %in% c("Administration", "Department", "White", "House",
            "CDC", "HHS", "DOJ", "DOE", "DOD", "Pentagon", "Congress",
            "Supreme", "Court", "Senate", "FDA", "NIH", "ICE", "NEA",
            "USAID", "PEPFAR", "FHWA", "SNAP", "NCAA", "NRA",
            "ACLU", "GLAD", "Lambda", "Harvard", "UCLA", "Church",
            "Republicans", "Democrats", "Smithsonian") ~ "ORG",
```

```

    # Locations (states, cities, countries)
    token %in% c("Washington", "America", "United", "States", "U.S.",
                "D.C.", "Massachusetts", "California", "Texas", "Idaho",
                "Connecticut", "Virginia", "Kansas", "Denver", "Maine",
                "Chicago", "Philadelphia", "Boston", "Providence",
                "Minneapolis", "Hungary", "Venezuela", "Salvador") ~ "GPE",

    # Default: classify remaining proper nouns as ORG
    # (most government entities in this dataset)
    TRUE ~ "ORG"
  )
) %>%
distinct(doc_id, token, entity) %>%
select(doc_id, token, entity)

# Extract monetary amounts using regex
money_pattern <- "\\$[0-9,]+(\\. [0-9]+)?(\\s)?(million|billion|trillion)?"

money_entities <- policy_data %>%
  mutate(
    money_matches = str_extract_all(content, money_pattern)
  ) %>%
  unnest(cols = money_matches) %>%
  select(doc_id = entry_id, token = money_matches) %>%
  mutate(
    doc_id = as.integer(doc_id),
    entity = "MONEY"
  ) %>%
  filter(!is.na(token)) %>%
  select(doc_id, token, entity)

# Combine proper nouns and money entities
entities <- bind_rows(
  proper_nouns,
  money_entities
) %>%
  arrange(doc_id, token)
}

cat("Entity extraction complete! Extracted", nrow(entities), "entity mentions.\n")
cat("Unique entities found:", n_distinct(entities$token), "\n")

entity_types <- entities %>% count(entity, sort = TRUE)
cat("\nEntity types:\n")
print(entity_types)
} else {
  cat("Skipping NER analysis - model not available.\n")
  entities <- tibble(doc_id = integer(), token = character(), entity = character())
}

```

```

## Running linguistic annotation on 368 entries...
## This may take 1-2 minutes. Please wait...
##
## Annotation complete!
## Model doesn't include NER. Using rule-based entity extraction...

```

```
##
## Entity extraction complete! Extracted 2552 entity mentions.
## Unique entities found: 901
##
## Entity types:
##   entity    n
## 1     ORG 1914
## 2 PERSON  439
## 3     GPE  168
## 4     MONEY 31
```

1.6.5 Entity Summary Statistics

```
if (nrow(entities) > 0) {
  # Count most common entities across all types
  entity_summary <- entities %>%
    count(token, entity, sort = TRUE) %>%
    head(30)

  kable(entity_summary, caption = "Top 30 Named Entities Detected",
        col.names = c("Entity", "Type", "Frequency"), format = "latex",
        booktabs = TRUE, longtable = TRUE) %>%
    kable_styling(latex_options = c("striped", "scale_down",
                                     "repeat_header"), font_size = 9)
}
```

Table 4: Top 30 Named Entities Detected

Entity	Type	Frequency
Trump	PERSON	183
President	PERSON	99
LGBTQ	ORG	78
Department	ORG	75
U.S.	GPE	67
Americans	ORG	38
Justice	PERSON	38
House	ORG	36
Court	ORG	35
National	ORG	32
White	ORG	32
Secretary	PERSON	31
Supreme	ORG	30
Health	ORG	26
Education	ORG	23
United	GPE	22
States	GPE	19
State	ORG	17
Biden	PERSON	16
HIV	ORG	14
Washington	GPE	14
Act	ORG	12
Administration	ORG	12
Council	ORG	12
America	GPE	11

Table 4: Top 30 Named Entities Detected (*continued*)

Entity	Type	Frequency
Congress	ORG	11
General	PERSON	11
Office	ORG	11
Defense	ORG	10
Donald	ORG	10

1.6.6 Entities by Type

```

if (nrow(entities) > 0) {
  # Analyze distribution of entity types
  entity_type_summary <- entities %>%
    count(entity, sort = TRUE)

  ggplot(entity_type_summary, aes(x = reorder(entity, n), y = n,
    fill = entity)) + geom_col(show.legend = FALSE) + coord_flip() +
    labs(title = "Named Entity Types Detected", subtitle = "Distribution of entity types
    ↪ across policy descriptions",
    x = "Entity Type", y = "Count") + theme_minimal() +
    theme(text = element_text(size = 12))

  kable(entity_type_summary, caption = "Entity Type Distribution",
    format = "latex", booktabs = TRUE, longtable = TRUE) %>%
    kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)
}

```

Table 5: Entity Type Distribution

entity	n
ORG	1914
PERSON	439
GPE	168
MONEY	31

1.6.7 Extract Organizations (Government Agencies)

```

if (nrow(entities) > 0) {
  # Extract all ORGANIZATION entities
  orgs <- entities %>%
    filter(entity == "ORG") %>%
    count(token, sort = TRUE) %>%
    head(20)

  if (nrow(orgs) > 0) {
    ggplot(orgs, aes(x = reorder(token, n), y = n)) + geom_col(fill = "steelblue") +
      coord_flip() + labs(title = "Top 20 Organizations Mentioned",
        subtitle = "Government agencies and institutions referenced in policy actions",
        x = "Organization", y = "Mentions") + theme_minimal()
  }
}

```

```

kable(orgs, caption = "Top 20 Organizations (Agencies, Institutions)",
      col.names = c("Organization", "Mentions"), format = "latex",
      booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)
} else {
  cat("No organizations extracted.\n")
}
}

```

Table 6: Top 20 Organizations (Agencies, Institutions)

Organization	Mentions
LGBTQ	78
Department	75
Americans	38
House	36
Court	35
National	32
White	32
Supreme	30
Health	26
Education	23
State	17
HIV	14
Act	12
Administration	12
Council	12
Congress	11
Office	11
Defense	10
Donald	10
president	10

1.6.8 Extract People (Government Officials)

```

if (nrow(entities) > 0) {
  # Extract all PERSON entities
  people <- entities %>%
    filter(entity == "PERSON") %>%
    count(token, sort = TRUE) %>%
    head(20)

  if (nrow(people) > 0) {
    ggplot(people, aes(x = reorder(token, n), y = n)) + geom_col(fill = "coral") +
      coord_flip() + labs(title = "Top 20 People Mentioned",
        subtitle = "Government officials and public figures referenced in policy actions",
        x = "Person", y = "Mentions") + theme_minimal()

    kable(people, caption = "Top 20 People (Officials, Public Figures)",
          col.names = c("Person", "Mentions"), format = "latex",
          booktabs = TRUE, longtable = TRUE) %>%
      kable_styling(latex_options = c("striped", "scale_down",
        "repeat_header"), font_size = 9)
  }
}

```



```

} else {
  cat("No people extracted.\n")
}
}

```

Table 7: Top 20 People (Officials, Public Figures)

Person	Mentions
Trump	183
President	99
Justice	38
Secretary	31
Biden	16
General	11
Attorney	7
Obama	7
DeVos	4
McEnany	4
Pence	4
Pompeo	4
Ellis	3
Hegseth	3
Kennedy	3
Leavitt	3
Carson	2
Graham	2
James	2
Robertson	2

###Extract Monetary Amounts

```

if (nrow(entities) > 0) {
  # Extract all MONEY entities
  money_entities <- entities %>%
    filter(entity == "MONEY") %>%
    count(token, sort = TRUE)

  if (nrow(money_entities) > 0) {
    kable(money_entities, caption = "Monetary Amounts Mentioned in Policy Actions",
          col.names = c("Amount", "Frequency"), format = "latex",
          booktabs = TRUE, longtable = TRUE) %>%
      kable_styling(latex_options = c("striped", "scale_down",
                                      "repeat_header"), font_size = 9)
  } else {
    cat("No monetary amounts detected.\n")
  }
}

```

Table 8: Monetary Amounts Mentioned in Policy Actions

Amount	Frequency
\$1.5 billion	2
\$8 million	2
\$1.1 billion	1

Table 8: Monetary Amounts Mentioned in Policy Actions (*continued*)

Amount	Frequency
\$1.25 million	1
\$1.4 billion	1
\$100,000	1
\$12 million	1
\$15 million	1
\$175 million	1
\$18 million	1
\$1million	1
\$2	1
\$2.9 billion	1
\$250 million	1
\$258 million	1
\$3 trillion	1
\$35,000,	1
\$40 million	1
\$5 million	1
\$510 million	1
\$58,000	1
\$584 million	1
\$5million	1
\$6 billion	1
\$600,000	1
\$7 billion	1
\$800 million	1
\$83 million	1
\$9 million	1

1.6.9 Extract Geographic Entities

```

if (nrow(entities) > 0) {
  # Extract all GPE (Geo-Political Entity) entities
  locations <- entities %>%
    filter(entity == "GPE") %>%
    count(token, sort = TRUE) %>%
    head(20)

  if (nrow(locations) > 0) {
    kable(locations, caption = "Top 20 Geographic Locations Mentioned",
          col.names = c("Location", "Mentions"), format = "latex",
          booktabs = TRUE, longtable = TRUE) %>%
      kable_styling(latex_options = c("striped", "scale_down",
                                      "repeat_header"), font_size = 9)
  } else {
    cat("No geographic locations extracted.\n")
  }
}

```

Table 9: Top 20 Geographic Locations Mentioned

Location	Mentions
U.S.	67
United States	22
Washington	19
America	14
California	11
Connecticut	7
D.C.	5
Minneapolis	5
Philadelphia	3
Salvador	2
Venezuela	2
Virginia	2
Boston	1
Chicago	1
Denver	1
Idaho	1
Kansas	1
Massachusetts	1
Texas	1

1.6.10 NER Insights Summary

```

if (nrow(entities) > 0) {
  cat("\n=== NER Analysis Summary ===\n\n")

  cat("Total entity mentions:", nrow(entities), "\n")
  cat("Unique entities:", n_distinct(entities$token), "\n\n")

  entity_counts <- entities %>%
    count(entity, sort = TRUE)

  cat("Breakdown by entity type:\n")
  for (i in 1:nrow(entity_counts)) {
    cat(sprintf("  %s: %d\n", entity_counts$entity[i], entity_counts$n[i]))
  }

  cat("\nKey Findings from NER:\n")
  cat("• NER automatically discovered", n_distinct(entities %>%
    filter(entity == "ORG") %>%
    pull(token)), "government agencies\n")
  cat("• Identified", n_distinct(entities %>%
    filter(entity == "PERSON") %>%
    pull(token)), "unique individuals mentioned\n")
  cat("• Found", n_distinct(entities %>%
    filter(entity == "GPE") %>%
    pull(token)), "geographic locations affected\n")
  cat("• This provides comprehensive coverage without manual keyword lists\n")
}

```

```

##
## === NER Analysis Summary ===

```

```
##
## Total entity mentions: 2552
## Unique entities: 901
##
## Breakdown by entity type:
##   ORG: 1914
##   PERSON: 439
##   GPE: 168
##   MONEY: 31
##
## Key Findings from NER:
## • NER automatically discovered 821 government agencies
## • Identified 31 unique individuals mentioned
## • Found 20 geographic locations affected
## • This provides comprehensive coverage without manual keyword lists
```

1.6.11 NER Methodology Note

```
cat("\n### Methodology Used\n\n")
```

```
##
## ### Methodology Used
```

```
if (exists("ner_results") && "entity" %in% colnames(ner_results) &&
    any(!is.na(ner_results$entity))) {
  cat("**Built-in NER**: This analysis used the built-in Named Entity Recognition from the
  ↳ udpipe model, which provides pre-trained entity classification.\n")
} else if (exists("entities") && nrow(entities) > 0) {
  cat("**Rule-based Entity Extraction**: Since the udpipe model doesn't include built-in NER,
  ↳ this analysis used:\n\n")
  cat("1. **Part-of-Speech tagging** to identify proper nouns (PROPN)\n")
  cat("2. **Pattern matching** to classify entities based on context and known keywords\n")
  cat("3. **Regular expressions** to extract monetary amounts\n\n")
  cat("**Note**: For more accurate NER, consider using the `spacyr` package (requires Python +
  ↳ spaCy) or other specialized NER tools.\n")
}
```

```
## **Rule-based Entity Extraction**: Since the udpipe model doesn't include built-in NER, this analysis use
##
## 1. **Part-of-Speech tagging** to identify proper nouns (PROPN)
## 2. **Pattern matching** to classify entities based on context and known keywords
## 3. **Regular expressions** to extract monetary amounts
##
## **Note**: For more accurate NER, consider using the `spacyr` package (requires Python + spaCy) or other
```

1.7 Step 7: Combined Analysis (Rule-Based + NER)

Analyzing patterns using both categorical features and extracted entities:

```
if (nrow(entities) > 0) {
  # Join policy categories with extracted organizations
  policy_with_entities <- policy_data_enhanced %>%
```

```

left_join(entities %>%
  filter(entity == "ORG") %>%
  select(doc_id, org = token), by = c(entry_id = "doc_id"))

# Which agencies appear most in each policy domain?
agency_by_domain <- policy_with_entities %>%
  filter(!is.na(org)) %>%
  count(policy_domain, org, sort = TRUE) %>%
  group_by(policy_domain) %>%
  slice_head(n = 5) %>%
  ungroup()

kable(agency_by_domain, caption = "Top 5 Agencies by Policy Domain",
  col.names = c("Policy Domain", "Agency", "Mentions"),
  format = "latex", booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)

# Which people are associated with which action types?
people_by_action <- policy_with_entities %>%
  left_join(entities %>%
    filter(entity == "PERSON") %>%
    select(doc_id, person = token), by = c(entry_id = "doc_id")) %>%
  filter(!is.na(person)) %>%
  count(action_type, person, sort = TRUE) %>%
  group_by(action_type) %>%
  slice_head(n = 3) %>%
  ungroup()

kable(people_by_action, caption = "Top People by Action Type",
  col.names = c("Action Type", "Person", "Mentions"), format = "latex",
  booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)
}

```

Table 10: Top People by Action Type

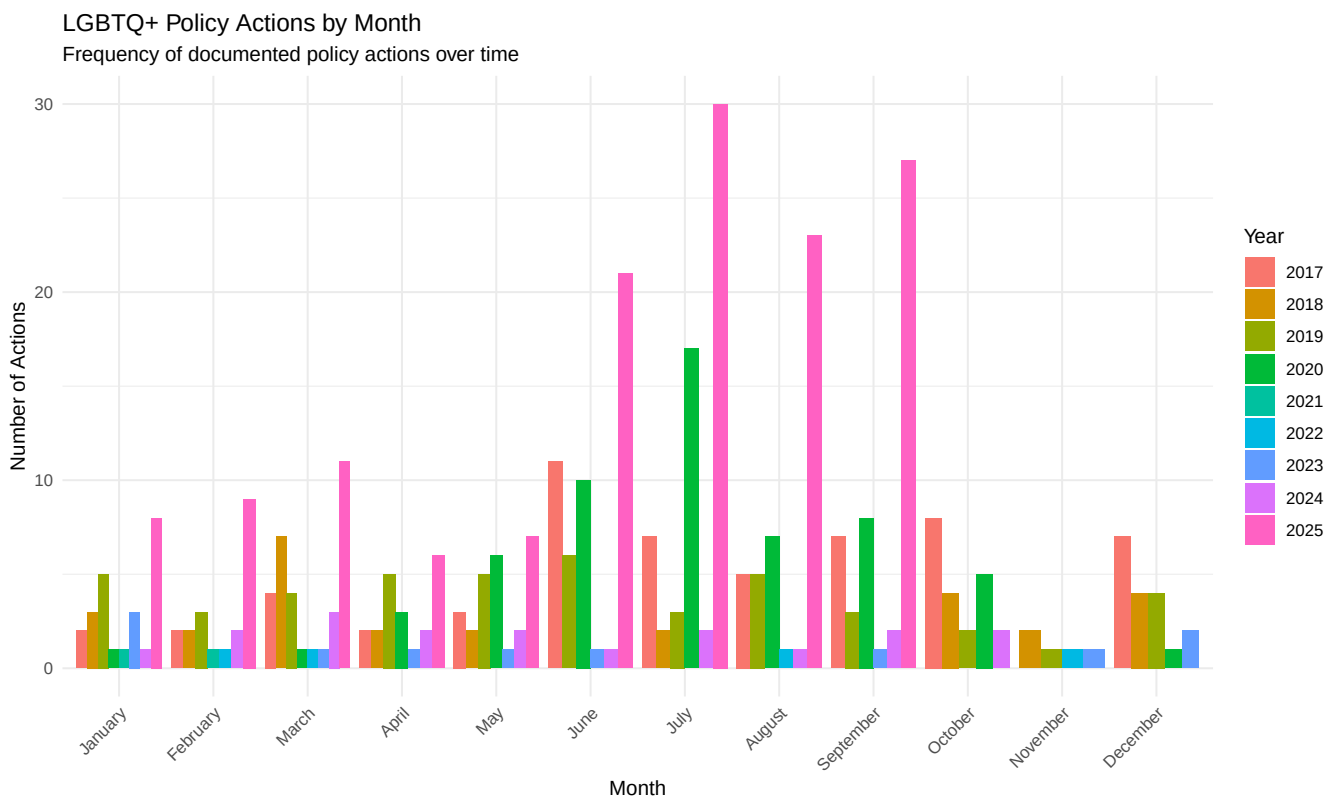
Action Type	Person	Mentions
Court Action	Trump	84
Court Action	President	53
Court Action	Kennedy	34
Executive Action	Trump	129
Executive Action	President	80
Executive Action	Secretary	53
Other	Trump	282
Other	President	177
Other	Secretary	79
Restriction/Removal	Trump	353
Restriction/Removal	President	137
Restriction/Removal	Justice	105
Rhetoric/Statement	Trump	140
Rhetoric/Statement	President	57
Rhetoric/Statement	Secretary	35

1.7.1 Step 8: Exploratory Data Analysis

Analyzing temporal and categorical patterns:

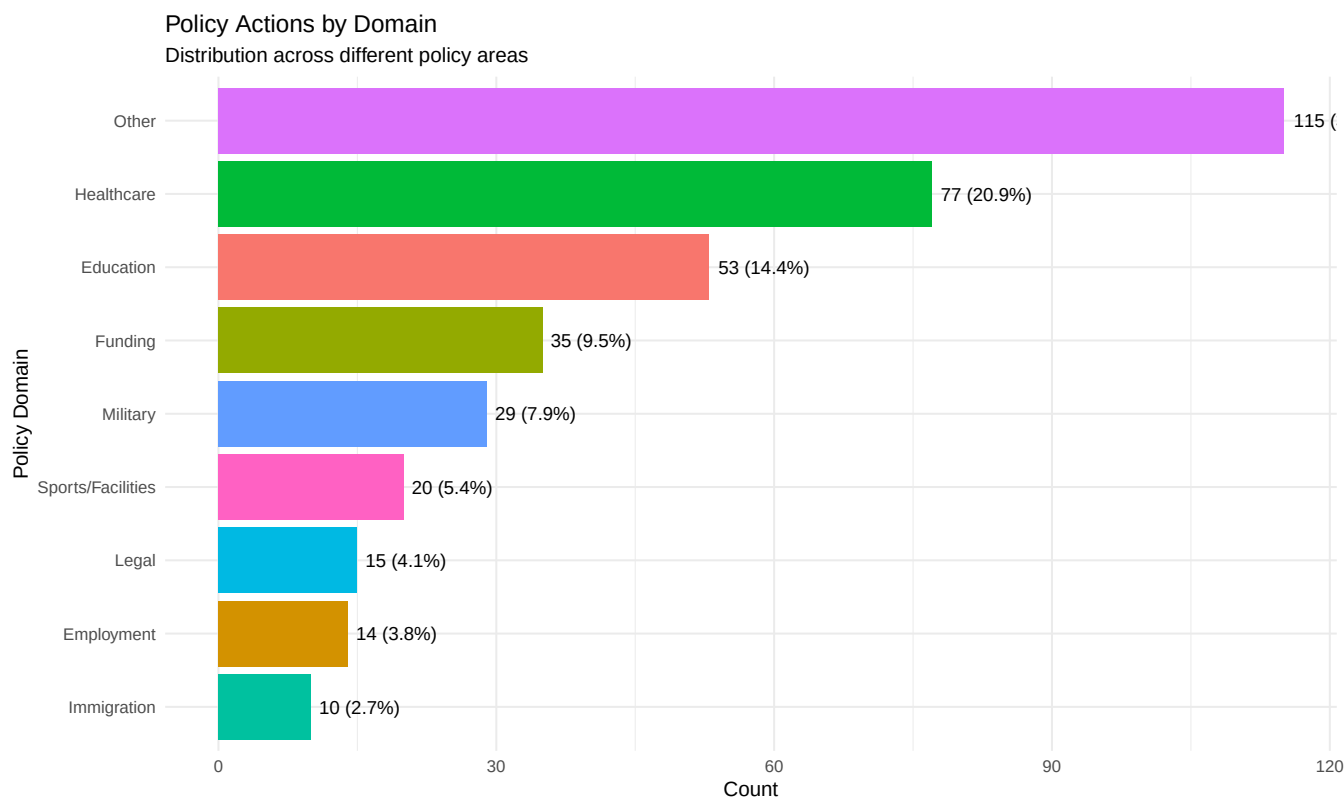
```
# Actions over time
temporal_summary <- policy_data_enhanced %>%
  group_by(year, month) %>%
  summarize(action_count = n(), .groups = "drop") %>%
  arrange(year, month)

# Visualize temporal trends
ggplot(temporal_summary, aes(x = month, y = action_count, fill = factor(year))) +
  geom_col(position = "dodge") + labs(title = "LGBTQ+ Policy Actions by Month",
  subtitle = "Frequency of documented policy actions over time",
  x = "Month", y = "Number of Actions", fill = "Year") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



```
# Policy domain distribution
domain_summary <- policy_data_enhanced %>%
  count(policy_domain, sort = TRUE) %>%
  mutate(percentage = n/sum(n) * 100)

ggplot(domain_summary, aes(x = reorder(policy_domain, n), y = n,
  fill = policy_domain)) + geom_col(show.legend = FALSE) +
  coord_flip() + labs(title = "Policy Actions by Domain", subtitle = "Distribution across
  ↪ different policy areas",
  x = "Policy Domain", y = "Count") + theme_minimal() + geom_text(aes(label = paste0(n,
  " (", round(percentage, 1), "%)")), hjust = -0.1, size = 3.5)
```



```
# Action type distribution
action_summary <- policy_data_enhanced %>%
  count(action_type, sort = TRUE) %>%
  mutate(percentage = n/sum(n) * 100)

kable(action_summary, caption = "Distribution of Action Types",
  col.names = c("Action Type", "Count", "Percentage"), format = "latex",
  booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)
```

Table 11: Distribution of Action Types

Action Type	Count	Percentage
Restriction/Removal	131	35.597826
Other	130	35.326087
Executive Action	48	13.043478
Rhetoric/Statement	42	11.413044
Court Action	17	4.619565

1.8 Step 9: Text Mining and Keyword Analysis

Identifying most common terms in policy descriptions:

```
# Tokenize and analyze words
policy_tokens <- policy_data_enhanced %>%
  select(entry_id, content) %>%
  unnest_tokens(word, content) %>%
```

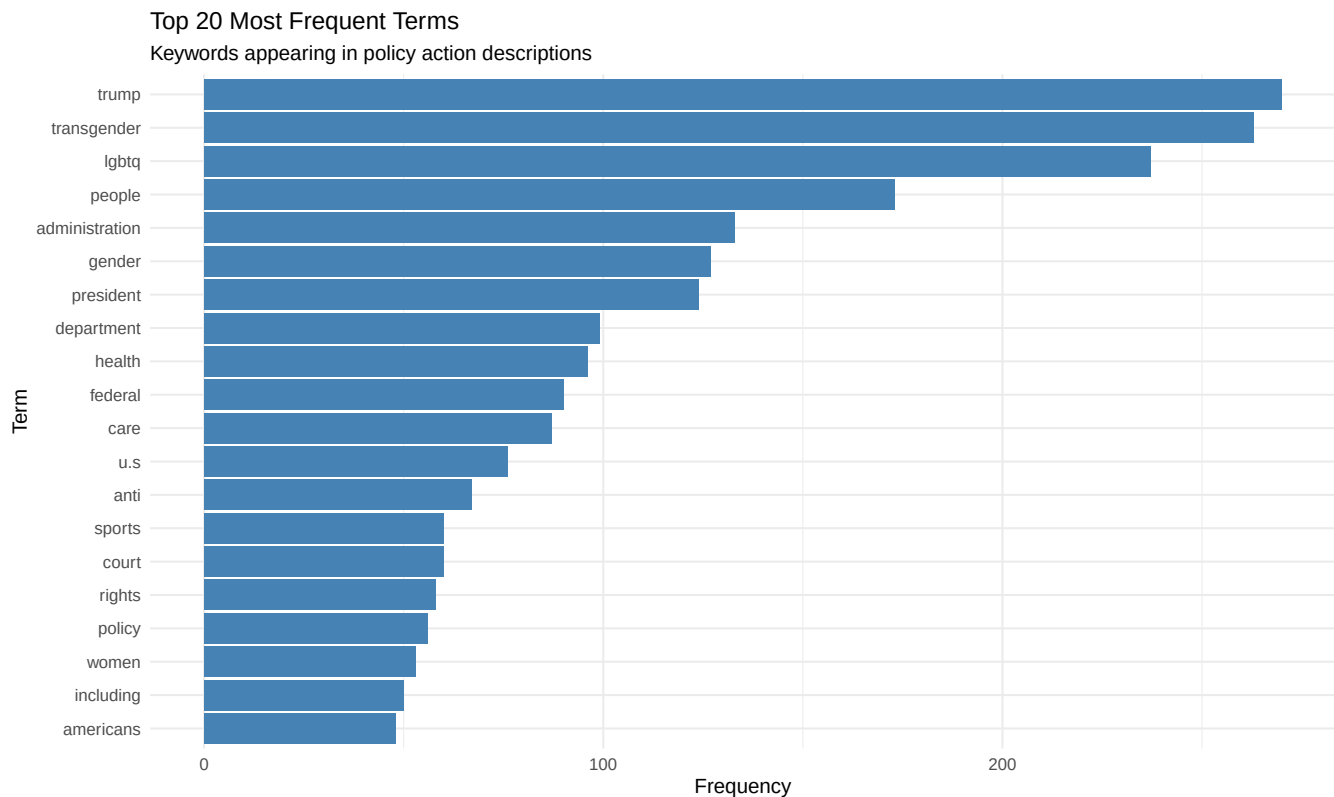
```

anti_join(stop_words, by = "word") %>%
filter(!str_detect(word, "^\\d+$")) # Remove numbers

# Top 20 most common words
word_frequency <- policy_tokens %>%
  count(word, sort = TRUE) %>%
  head(20)

ggplot(word_frequency, aes(x = reorder(word, n), y = n)) + geom_col(fill = "steelblue") +
  coord_flip() + labs(title = "Top 20 Most Frequent Terms",
  subtitle = "Keywords appearing in policy action descriptions",
  x = "Term", y = "Frequency") + theme_minimal()

```



```

# Display table
kable(word_frequency, caption = "Most Common Terms in Policy Descriptions",
  col.names = c("Term", "Frequency"), format = "latex", booktabs = TRUE,
  longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
  "repeat_header"), font_size = 9)

```

Table 12: Most Common Terms in Policy Descriptions

Term	Frequency
trump	270
transgender	263
lgbtq	237
people	173
administration	133

Table 12: Most Common Terms in Policy Descriptions (*continued*)

Term	Frequency
gender	127
president	124
department	99
health	96
federal	90
care	87
u.s	76
anti	67
court	60
sports	60
rights	58
policy	56
women	53
including	50
americans	48

1.9 Step 10: Key Findings and Insights

```

# Generate summary statistics
total_actions <- nrow(policy_data_enhanced)
date_range <- paste(format(min(policy_data_enhanced$date), "%B %d, %Y"),
  "to", format(max(policy_data_enhanced$date), "%B %d, %Y"))

# Calculate key metrics
restrictions <- policy_data_enhanced %>%
  filter(action_type == "Restriction/Removal") %>%
  nrow()

healthcare_impacts <- policy_data_enhanced %>%
  filter(mentions_healthcare) %>%
  nrow()

youth_impacts <- policy_data_enhanced %>%
  filter(mentions_youth) %>%
  nrow()

military_impacts <- policy_data_enhanced %>%
  filter(mentions_military) %>%
  nrow()

# NER-based counts
if (nrow(entities) > 0) {
  unique_orgs <- n_distinct(entities %>%
    filter(entity == "ORG") %>%
    pull(token))
  unique_people <- n_distinct(entities %>%
    filter(entity == "PERSON") %>%
    pull(token))
  unique_locations <- n_distinct(entities %>%
    filter(entity == "GPE") %>%
    pull(token))
} else {

```

```

unique_orgs <- NA
unique_people <- NA
unique_locations <- NA
}

```

1.10 Summary Statistics

Total Documented Actions: “368” Date Range: “January 20, 2017 to September 24, 2025” Restriction/Removal Actions: 131 (35.6%) Actions Affecting Healthcare: 83 (22.6%) Actions Affecting Youth: 82 (22.3%) Actions Affecting Military: 29 (7.9%) Named Entity Recognition Results Government Agencies Identified: “821” Officials/People Mentioned: “31” Geographic Locations Affected: “20” Key Insights Healthcare and Funding are the most frequently targeted policy domains Restriction/Removal actions constitute the majority of policy changes Actions disproportionately affect vulnerable populations, especially youth Multiple federal agencies (DOE, CDC, DOJ, HHS, DOD) are involved across entries Court interventions frequently occur to block administration actions NER revealed extensive involvement of specific agencies and officials not captured by keyword matching

1.11 Step 11: Save Enhanced Data to Database

```

# Write enhanced data to PostgreSQL
dbWriteTable(conn = con, name = Id(schema = "public", table = "lgbtq_policy_enhanced"),
  value = policy_data_enhanced, overwrite = TRUE, row.names = FALSE)

# Write NER entities to database
if (nrow(entities) > 0) {
  dbWriteTable(conn = con, name = Id(schema = "public", table = "policy_entities"),
    value = entities, overwrite = TRUE, row.names = FALSE)

  # Create index for faster joins
  dbExecute(con, "CREATE INDEX IF NOT EXISTS idx_entities_doc
    ON public.policy_entities (doc_id);")
}

```

```
## [1] 0
```

```
cat("Enhanced data and NER results saved to database.\n")
```

```
## Enhanced data and NER results saved to database.
```

1.12 Step 12: Final Database Verification

```

# Final query to verify all data in database
final_check <- dbGetQuery(con, "
  SELECT
    COUNT(*) as total_records,
    MIN(date) as earliest_date,
    MAX(date) as latest_date,
    COUNT(DISTINCT EXTRACT(YEAR FROM date)) as years_covered
  FROM public.lgbtq_policy_enhanced;
")

```

```
kable(final_check, caption = "Final Database Verification", format = "latex",
      booktabs = TRUE, longtable = TRUE) %>%
  kable_styling(latex_options = c("striped", "scale_down",
    "repeat_header"), font_size = 9)
```

Table 13: Final Database Verification

total_records	earliest_date	latest_date	years_covered
368	2017-01-20	2025-09-24	9

```
# Check NER entities table
if (dbExistsTable(con, Id(schema = "public", table = "policy_entities"))) {
  entity_check <- dbGetQuery(con, "
  SELECT
    entity,
    COUNT(*) as count
  FROM public.policy_entities
  GROUP BY entity
  ORDER BY count DESC;
  ")

  kable(entity_check, caption = "Entity Types in Database",
        format = "latex", booktabs = TRUE, longtable = TRUE) %>%
    kable_styling(latex_options = c("striped", "scale_down",
      "repeat_header"), font_size = 9)
}
```

Table 14: Entity Types in Database

entity	count
ORG	1914
PERSON	439
GPE	168
MONEY	31

```
# Close database connection
dbDisconnect(con)
cat("\nDatabase connection closed.\n")
```

```
##
## Database connection closed.
```

1.13 Reproducibility Notes

1.13.1 Environment Setup for DDEV

To reproduce this analysis with DDEV PostgreSQL:

1. Start DDEV: `ddev start`
2. Ensure PostgreSQL is running on port 55432 (configured in `.ddev/docker-compose.postgres.yaml`)
3. Run this R Markdown: All data extraction, transformation, and loading will execute automatically

1.13.2 Required R Packages

```
# Set CRAN mirror first
options(repos = c(CRAN = "https://cran.rstudio.com/"))

# Install packages (only run if needed)
install.packages(c("tidyverse", "lubridate", "DBI", "RPostgres",
  "officer", "xml2", "tidytext", "knitr", "kableExtra", "udpipe",
  "formatR"))
```

1.13.3 One-Time NER Model Download

```
library(udpipe)
udmodel_download <- udpipe_download_model(language = "english")
```

1.14 Data Source

Original data: Untidy.docx containing chronological Trump Administration LGBTQ+ policy actions from GLAAD's accountability project. Drawn from <https://glaad.org/trump-accountability-tracker/>

1.15 Methodological Approach

This analysis combines:

1. Rule-based categorization (fast, domain-specific)
2. Machine learning NER (comprehensive, discovers new entities)
3. Text mining (frequency and keyword analysis)
4. Temporal analysis (trends over time)

1.16 Session Information

```
sessionInfo()
```

```
## R version 4.5.1 (2025-06-13)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sequoia 15.1
##
## Matrix products: default
## BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK versi
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
```

```
##
## other attached packages:
## [1] udpipe_0.8.12      kableExtra_1.4.0  knitr_1.50        scales_1.4.0
## [5] tidytext_0.4.3     xml2_1.4.0        officer_0.7.0     RPostgres_1.4.8
## [9] DBI_1.2.3          lubridate_1.9.4   forcats_1.0.0     stringr_1.5.2
## [13] dplyr_1.1.4        purrr_1.1.0       readr_2.1.5       tidyr_1.3.1
## [17] tibble_3.3.0       ggplot2_4.0.0     tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] gtable_0.3.6       xfun_0.53         lattice_0.22-7     tzdb_0.5.0
## [5] vctrs_0.6.5        tools_4.5.1       generics_0.1.4     blob_1.2.4
## [9] janeaustenr_1.0.0  pkgconfig_2.0.3   tokenizers_0.3.0   Matrix_1.7-4
## [13] data.table_1.17.8  RColorBrewer_1.1-3 S7_0.2.0           uuid_1.2-1
## [17] lifecycle_1.0.4    compiler_4.5.1    farver_2.1.2       textshaping_1.0.3
## [21] htmltools_0.5.8.1  SnowballC_0.7.1   yaml_2.3.10        pillar_1.11.1
## [25] openssl_2.3.3      tidyselect_1.2.1  zip_2.3.3          digest_0.6.37
## [29] stringi_1.8.7       labeling_0.4.3     fastmap_1.2.0      grid_4.5.1
## [33] cli_3.6.5          magrittr_2.0.4     withr_3.0.2        bit64_4.6.0-1
## [37] timechange_0.3.0    rmarkdown_2.29    bit_4.6.0          askpass_1.2.1
## [41] ragg_1.5.0         hms_1.1.3         evaluate_1.0.5     viridisLite_0.4.2
## [45] rlang_1.1.6        Rcpp_1.1.0        glue_1.8.0         formatR_1.14
## [49] svglite_2.2.1      rstudioapi_0.17.1 R6_2.6.1           systemfonts_1.2.3
```

1.17 References

Wickham, H., et al. (2019). Welcome to the tidyverse. *Journal of Open Source Software*. Golemund, G. & Wickham, H. (2011). Dates and Times Made Easy with lubridate. *Journal of Statistical Software*. Silge, J. & Robinson, D. (2016). tidytext: Text Mining and Analysis Using Tidy Data Principles in R. Wijffels, J. (2021). udpipe: Tokenization, Parts of Speech Tagging, Lemmatization and Dependency Parsing with the ‘UDPipe’ ‘NLP’ Toolkit. R package.